X T E R R A R O B O T I C S

Orion Sensor Streaming System

Software Installation Guide

NVIDIA Jetson Orin Nano Super • ROS2 Humble • Docker • CycloneDDS

C O N F I D E N T I A L — F O R C U S T O M E R U S E O N L Y

1. Overview

This guide provides step-by-step instructions for setting up the xTerra Orion Navigation streaming environment on the NVIDIA Jetson Orin Nano Super. Following this guide will result in a fully operational Docker-based ROS2 environment capable of streaming Livox MID-360 LiDAR and Intel RealSense D435i camera data across the network.

The installation is structured into the following phases:

- Phase 1 — Host machine preparation (Docker permissions and udev rules)
- Phase 2 — Repository cloning and CycloneDDS network configuration
- Phase 3 — Docker container build and launch
- Phase 4 — In-container software installation (Livox, RealSense SDK, ROS2 driver)
- Phase 5 — Starting the sensor streaming pipeline

> ℹ️ **NOTE**
> Commands in Section 2 must be executed on the Jetson HOST machine — not inside any Docker container. Commands from Section 4 onwards are executed inside the container.

2. Host Machine Preparation

2.1 Docker Group Permissions

By default, running Docker commands requires sudo privileges. The following grants the current user permission to run Docker without sudo.

1 **Add user to the Docker group and activate**

Terminal
```
sudo usermod -aG docker $USER
newgrp docker
```

The -aG flag appends the user to the docker group without removing existing group memberships. The newgrp command activates the change immediately in the current session. For a permanent effect across all sessions, log out and back in, or reboot the system.

2.2 Intel RealSense udev Rules

Install the official Intel RealSense udev rules on the host:

2 **Download, install, and apply udev rules**

Terminal

```
sudo wget https://raw.githubusercontent.com/IntelRealSense/librealsense/master/
config/99-realsense-libusb.rules -O /etc/udev/rules.d/99-realsense-libusb.rules

sudo udevadm control --reload-rules
sudo udevadm trigger

# Verify the file is in place
ls -la /etc/udev/rules.d/ | grep realsense
```

⚠ **WARNING**

CRITICAL: After running the above commands, physically unplug and replug the RealSense USB cable. udev rules only take effect when the device is reconnected.

3. Repository Setup and Network Configuration

3.1 Clone the Repository

Clone the Orion Navigation repository to the recommended workspace location on the Jetson host:

3 **Create workspace and clone the repository**

Terminal

```
mkdir -p ~/dev/nav
cd ~/dev/nav
git clone https://github.com/xterra-robotics/orion_nav.git
cd orion_nav
git checkout sudden_demo
```

3.2 Configure LiDAR Network Interface

The Livox MID-360 LiDAR communicates over Ethernet. Assign the correct IP address to the Ethernet interface before launching the container:

4 **Assign IP address to the LiDAR Ethernet interface**

Terminal

```
sudo ip addr add 192.168.1.0/24 dev enP8p1s0
```

3.3 Configure CycloneDDS Network (cyclonedds.xml)

CycloneDDS uses an XML configuration file to determine which network interfaces and peer devices to use for ROS2 topic discovery. This file must be updated to match your specific network setup before launching the container.

5 Open the configuration file for editing

Terminal
```
nano ~/dev/nav/orion_nav/cyclonedds.xml
```

> **NOTE**
> If nano is not installed on the system, use vim instead: vim ~/dev/nav/orion_nav/cyclonedds.xml

Once open, make the following changes:

- NetworkInterface name — verify the WiFi interface name matches your Jetson's WiFi adapter. To confirm, run ifconfig on the Jetson host outside the container and look for an entry beginning with wl (e.g., wlP1p1s0).
- Peer address (laptop IP) — replace the existing laptop peer entry with your laptop's current WiFi IP address. Run ifconfig on the laptop and look under wlo1 or wlan0 to find it.

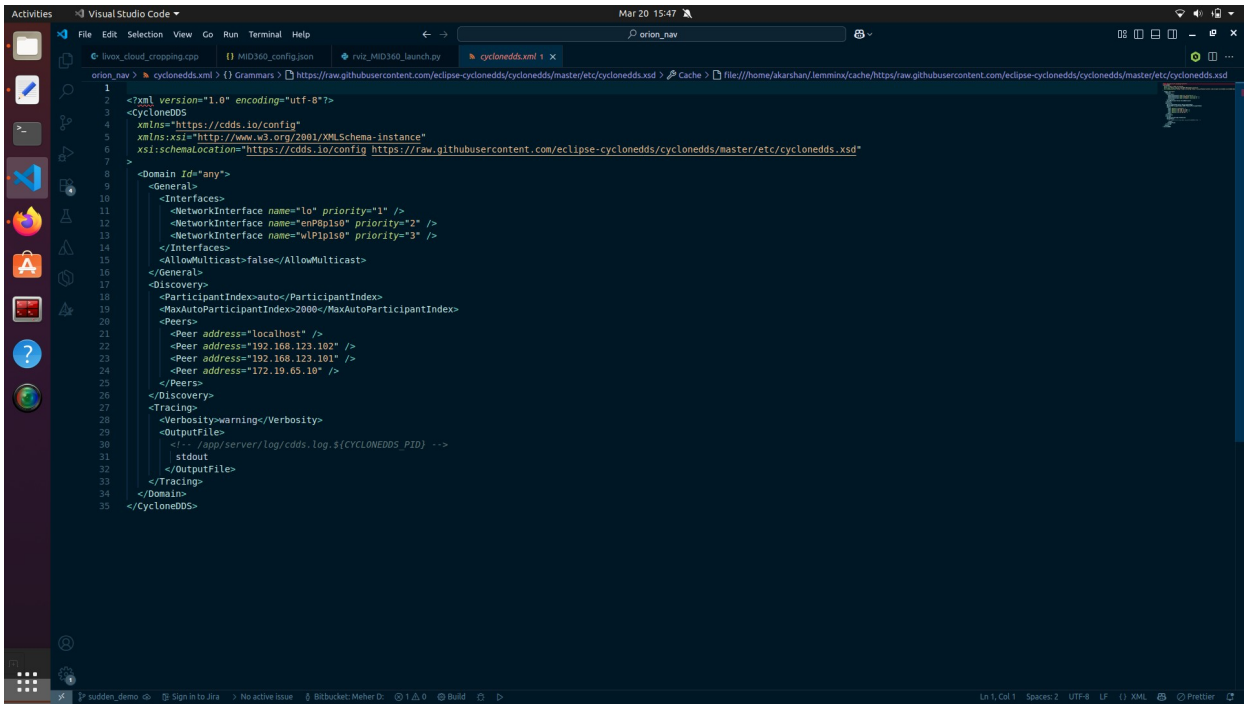

Figure 1 — cyclonedds.xml open in VS Code. The NetworkInterface entries and Peer IP addresses highlighted must be updated to match your network.

> **NOTE**
> The WiFi interface name on the Jetson (e.g., wlP1p1s0) must appear in the NetworkInterfaces block. If the name is wrong, ROS2 nodes will not communicate over WiFi. Cross-check with ifconfig on the Jetson host before proceeding.

4. Build and Launch the Docker Container

4.1 Run the Setup Script

The setup_autonomy.sh script builds the Docker image and starts the container. Passing true as an argument triggers a fresh Docker build.

6 **Build and launch the container**

Terminal
```
cd ~/dev/nav/orion_nav
./setup_autonomy.sh true
```

The build process may take several minutes on first run as it downloads and compiles all base dependencies.

4.2 Manual Container Attachment (if required)

If the setup script completes without dropping you into the container shell, start and attach to it manually using the following commands:

7 **Start and attach to container (only if not already inside)**

Terminal
```
docker compose up -d
docker exec -it xterra_autonomy_container bash
```

> ℹ️ **NOTE**
> If you are already inside the container after running setup_autonomy.sh, skip the above two commands. You can confirm you are inside the container by checking the shell prompt — it will show root@xterra-orin:/home/ros2_ws#

5. In-Container Software Installation

All commands in this section are executed inside the Docker container. Confirm you are inside the container before proceeding.

5.1 Livox SDK Installation

Install the Livox SDK and the corresponding ROS2 driver required for the MID-360 LiDAR:

8 **Run the Livox installation script**

Terminal
```
cd src/
./livox.sh
cd ..
```

> ℹ️ **NOTE**
> If you encounter any permission-related errors while running the script, use: sudo ./livox.sh

5.2 Configure ROS2 Environment Variables

Source the ROS2 workspace and set all required environment variables, including the CycloneDDS URI and ROS domain configuration:

9 **Source the environment setup script**

Terminal
```
source set_cyclonedds_ros_custom.sh
```

> ℹ️ **NOTE**
> This script sources the ROS2 workspace and sets RMW_IMPLEMENTATION, CYCLONEDDS_URI, and ROS_DOMAIN_ID. It must be run in every new container session before launching any ROS2 nodes.

5.3 Intel RealSense SDK Installation

Install the Intel RealSense SDK inside the container. The script builds librealsense v2.54.2 from source with FORCE_LIBUVC=ON, which is required for stable operation on ARM64/Jetson platforms:

10 **Run the RealSense installation script**

Terminal
```
./install_realsense.sh
```

The script performs the following automatically:

- Removes any conflicting pre-installed librealsense packages
- Builds librealsense v2.54.2 from source with FORCE_LIBUVC=ON
- Installs udev rules inside the container
- Verifies camera detection using rs-enumerate-devices

> ⚠️ **WARNING**

If the verification step reports 'Camera not detected', physically unplug and replug the RealSense USB cable on the host, then run rs-enumerate-devices manually inside the container to confirm detection.

5.4 Build the RealSense ROS2 Driver

Build the RealSense ROS2 camera driver packages from source to match the installed SDK version:

11 **Build RealSense ROS2 packages**

Terminal
```
colcon build --packages-select realsense2_camera_msgs realsense2_camera
realsense2_description --cmake-args -DCMAKE_BUILD_TYPE=Release -DFORCE_LIBUVC=ON
```

6. Start Sensor Streaming

With all software installed and the environment configured, launch the full sensor streaming pipeline:

12 **Start the LiDAR and camera streaming pipeline**

Terminal
```
./start_streaming.sh
```

This script launches both the Livox MID-360 LiDAR and Intel RealSense D435i camera ROS2 nodes. Once running, all sensor topics are available on the network for any ROS2 client sharing the same CycloneDDS domain and peer configuration.

© 2026 xTerra Robotics. Confidential.